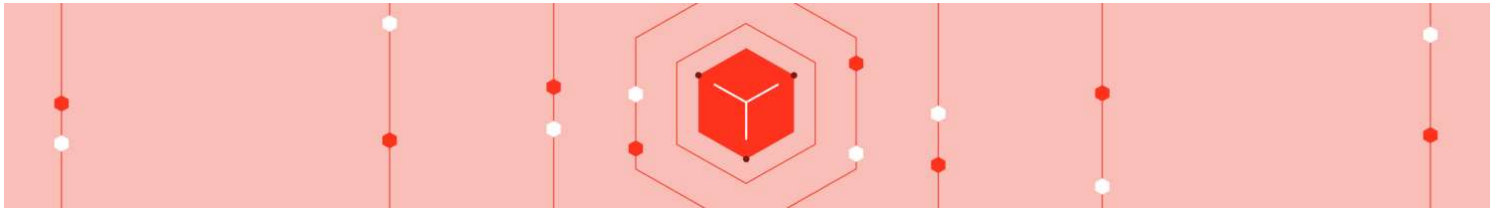


Categories



Diving Into Delta Lake: Unpacking The Transaction Log



by [Burak Yavuz](#), [Michael Armbrust](#) and [Brenner Heintz](#)

August 21, 2019 in [Company Blog](#)

Share this post



DELTA LAKE

The transaction log is key to understanding [Delta Lake](#) because it is the common thread that runs through many of its most important features, including ACID transactions, scalable metadata handling, time travel, and more. In this article, we'll explore what the Delta Lake transaction log is, how it works at the file level, and how it offers an elegant solution to the problem of multiple concurrent reads and writes.

The Delta Lake transaction log (also known as the DeltaLog) is an ordered record of every transaction that has ever been performed on a Delta Lake table since its inception.

What Is the Transaction Log Used For?

Single Source of Truth

Delta Lake is built on top of Apache Spark™ in order to allow multiple readers and writers of a given table to all work on the table at the same time. In order to show users correct views of the data at all times, the Delta Lake transaction log serves as a **single source of truth** – the central repository that tracks all changes that users make to the table.

When a user reads a Delta Lake table for the first time or runs a new query on an open table that has been modified since the last time it was read, **Spark checks the transaction log to see what new transactions have posted to the table, and then updates the end user's table with those new changes.** This ensures that a user's version of a table is always synchronized with the master record as of the most recent query, and that users cannot make divergent, conflicting changes to a table.

The Implementation of Atomicity on Delta Lake

One of the four properties of ACID transactions, **atomicity**, guarantees that operations (like an INSERT or UPDATE) performed on your [data lake](#) either complete fully, or don't complete at all. Without this property, it's far too easy for a hardware failure or a software bug to cause data to be only partially written to a table, resulting in messy or corrupted data.

The transaction log is the mechanism through which Delta Lake is able to offer the guarantee of atomicity. For all intents and purposes, if it's not recorded in the transaction log, it never happened. By only recording transactions that execute fully and completely, and using that record as the single source of truth, the Delta Lake transaction log allows users to reason about their data, and have peace of mind about its fundamental trustworthiness, at petabyte scale.

How Does the Transaction Log Work?

Breaking Down Transactions Into Atomic Commits

of one or more of the **actions** below.

- **Add file** – adds a data file.
- **Remove file** – removes a data file.
- **Update metadata** – Updates the table's metadata (e.g., changing the table's name, schema or partitioning).
- **Set transaction** – Records that a structured streaming job has committed a micro-batch with the given ID.
- **Change protocol** – enables new features by switching the Delta Lake transaction log to the newest software protocol.
- **Commit info** – Contains information around the commit, which operation was made, from where and at what time.

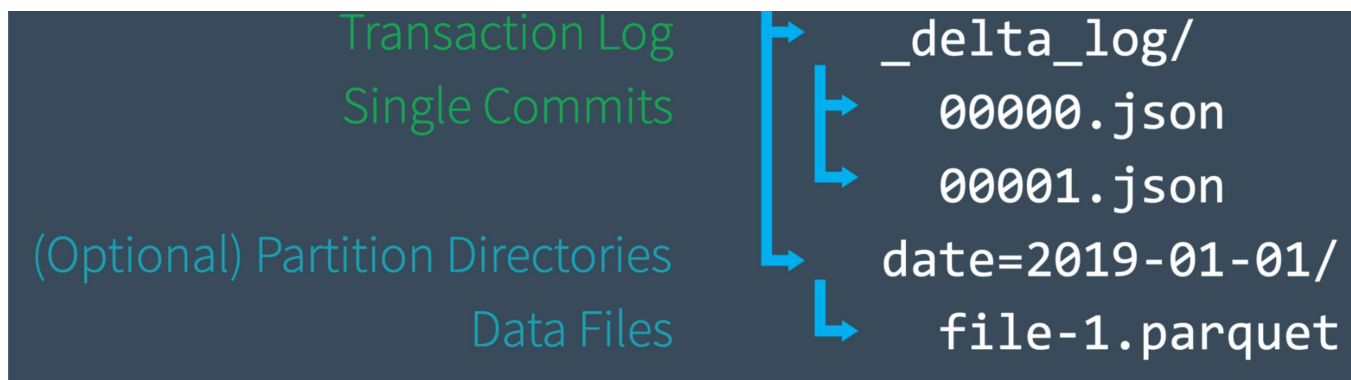
Those actions are then recorded in the transaction log as ordered, atomic units known as **commits**.

For example, suppose a user creates a transaction to add a new column to a table plus add some more data to it. Delta Lake would break that transaction down into its component parts, and once the transaction completes, add them to the transaction log as the following commits:

1. Update metadata – change the schema to include the new column
2. Add file – for each new file added

The Delta Lake Transaction Log at the File Level

When a user creates a Delta Lake table, that table's transaction log is automatically created in the `_delta_log` subdirectory. As he or she makes changes to that table, those changes are recorded as ordered, atomic commits in the transaction log. Each commit is written out as a JSON file, starting with `000000.json`. Additional changes to the table generate subsequent JSON files in ascending numerical order so that the next commit is written out as `000001.json`, the following as `000002.json`, and so on.



So, as an example, perhaps we might add additional records to our table from the data files 1.parquet and 2.parquet. That transaction would automatically be added to the transaction log, saved to disk as commit 000000.json. Then, perhaps we change our minds and decide to remove those files and add a new file instead (3.parquet). Those actions would be recorded as the next commit in the transaction log, as 000001.json, as shown below.

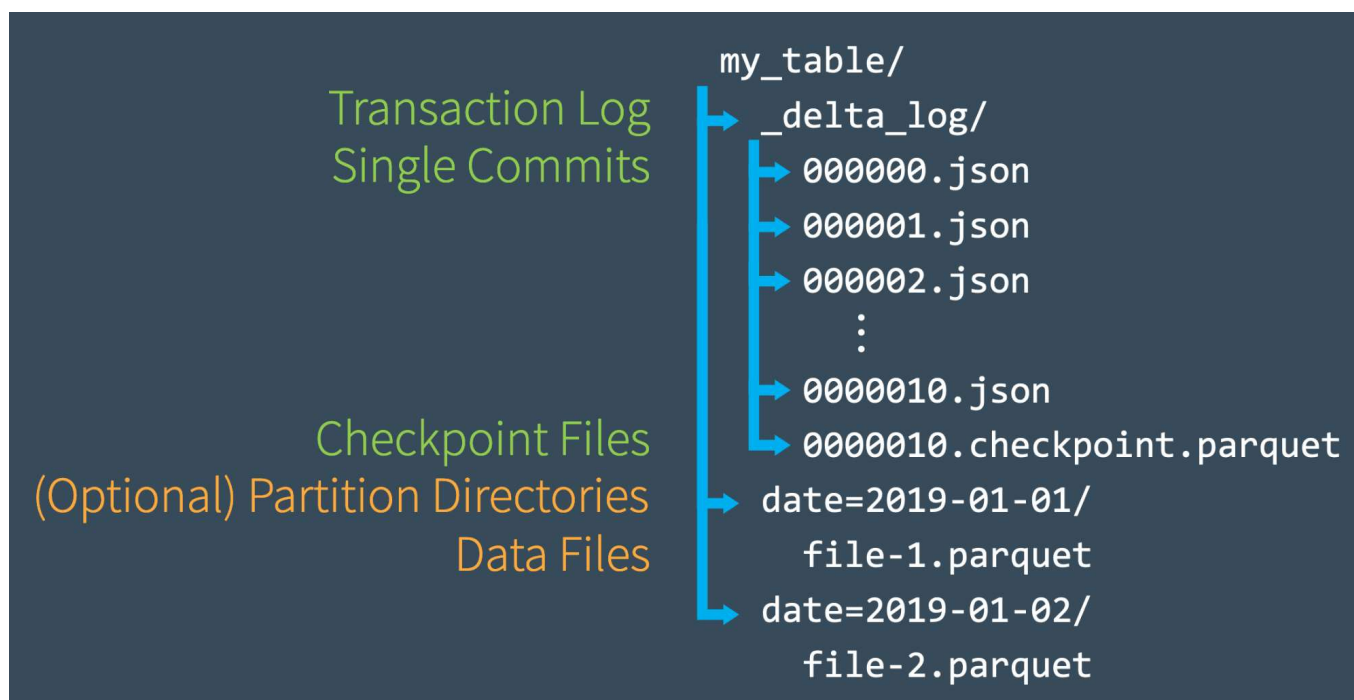


Even though 1.parquet and 2.parquet are no longer part of our Delta Lake table, their addition and removal are still recorded in the transaction log because those operations were performed on our table – despite the fact that they ultimately canceled each other out. **Delta Lake still retains atomic commits like these to ensure that in the event we need to audit our table or use “time travel” to see what our table looked like at a given point in time, we could do so accurately.**

using `VACUUM`.

Quickly Recomputing State With Checkpoint Files

Once we've made several commits to the transaction log, Delta Lake saves a checkpoint file in Parquet format in the same `_delta_log` subdirectory. **Delta Lake automatically generates checkpoint as needed to maintain good read performance.**



These checkpoint files save the entire state of the table at a point in time – in native Parquet format that is quick and easy for Spark to read. In other words, they offer the Spark reader a sort of “shortcut” to fully reproducing a table’s state that allows Spark to avoid reprocessing what could be thousands of tiny, inefficient JSON files.

To get up to speed, Spark can run a `listFrom` operation to view all the files in the transaction log, quickly skip to the newest checkpoint file, and only process those JSON commits made since the most recent checkpoint file was saved.

To demonstrate how this works, imagine that we’ve created commits all the way through `000007.json` as shown in the diagram below. Spark is up to speed through this commit, having automatically cached the most recent version of the table in memory. In the meantime, though, several other writers (perhaps your overly eager teammates) have written new data to the table, adding commits all the way through `000012.json`.



Rather than processing all of the intermediate JSON files, Spark can skip ahead to the most recent checkpoint file, since it contains the entire state of the table at commit #10. Now, Spark only has to perform incremental processing of `0000011.json` and `0000012.json` to have the current state of the table. Spark then caches version 12 of the table in memory. By following this workflow, Delta Lake is able to use Spark to keep the state of a table updated at all times in an efficient manner.

Dealing With Multiple Concurrent Reads and Writes

Now that we understand how the Delta Lake transaction log works at a high level, let's talk about concurrency. So far, our examples have mostly covered scenarios in which users commit transactions linearly, or at least without conflict. But what happens when Delta Lake is dealing with multiple concurrent reads and writes?

The answer is simple. **Since Delta Lake is powered by Apache Spark, it's not only possible for multiple users to modify a table at once – it's expected. To handle these situations, Delta Lake employs *optimistic concurrency control*.**

What Is Optimistic Concurrency Control?

Optimistic concurrency control is a method of dealing with concurrent transactions that assumes that transactions (changes) made to a table by different users can complete without conflicting with one another. It is incredibly fast because when dealing with petabytes of data, there's a high likelihood that users will be working on different parts of

For example, imagine that you and I are working on a jigsaw puzzle together. As long as we're both working on different parts of it – you on the corners, and me on the edges, for example – there's no reason why we can't each work on our part of the bigger puzzle at the same time, and finish the puzzle twice as fast. It's only when we need the same pieces, at the same time, that there's a conflict. That's optimistic concurrency control.

Of course, even with optimistic concurrency control, sometimes users do try to modify the same parts of the data at the same time. Luckily, Delta Lake has a protocol for that.

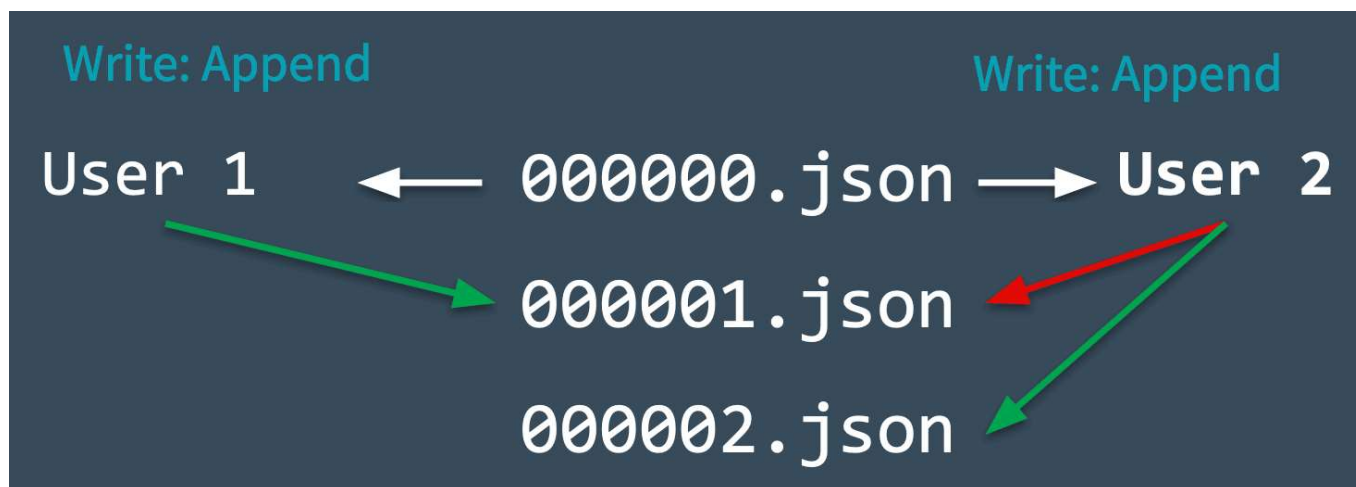
Solving Conflicts Optimistically

In order to offer ACID transactions, Delta Lake has a protocol for figuring out how commits should be ordered (known as the concept of **serializability** in databases), and determining what to do in the event that two or more commits are made at the same time. Delta Lake handles these cases by implementing a rule of **mutual exclusion**, then attempting to solve any conflict optimistically. This protocol allows Delta Lake to deliver on the ACID principle of **isolation**, which ensures that the resulting state of the table after multiple, concurrent writes is the same as if those writes had occurred serially, in isolation from one another.

In general, the process proceeds like this:

1. **Record the starting table version.**
2. **Record reads/writes.**
3. **Attempt a commit.**
4. **If someone else wins, check whether anything you read has changed.**
5. **Repeat.**

To see how this all plays out in real time, let's take a look at the diagram below to see how Delta Lake manages conflicts when they do crop up. Imagine that two users read from the same table, then each go about attempting to add some data to it.



- Delta Lake records the starting table version of the table (version 0) that is read prior to making any changes.
- Users 1 and 2 both attempt to append some data to the table at the same time. Here, we've run into a conflict because only one commit can come next and be recorded as 000001.json.
- Delta Lake handles this conflict with the concept of "mutual exclusion," which means that only one user can successfully make commit 000001.json. User 1's commit is accepted, while User 2's is rejected.
- Rather than throw an error for User 2, Delta Lake prefers to handle this conflict *optimistically*. It checks to see whether any new commits have been made to the table, and updates the table silently to reflect those changes, then simply retries User 2's commit on the newly updated table (without any data processing), successfully committing 000002.json.

In the vast majority of cases, this reconciliation happens silently, seamlessly, and successfully. However, in the event that there's an irreconcilable problem that Delta Lake cannot solve optimistically (for example, if User 1 deleted a file that User 2 also deleted), the only option is to throw an error.

As a final note, since all of the transactions made on Delta Lake tables are stored directly to disk, this process satisfies the ACID property of **durability**, meaning it will persist even in the event of system failure.

Other Use Cases

Time Travel

instruction guide, detailing exactly how to get from the table's original state to its current state.

Therefore, we can recreate the state of a table at any point in time by starting with an original table, and processing only commits made prior to that point. This powerful ability is known as "time travel," or data versioning, and can be a lifesaver in any number of situations. For more information, read the blog post [Introducing Delta Time Travel for Large Scale Data Lakes](#), or refer to the [Delta Lake time travel documentation](#).

Data Lineage and Debugging

As the definitive record of every change ever made to a table, the Delta Lake transaction log offers users a verifiable data lineage that is useful for governance, audit and compliance purposes. It can also be used to trace the origin of an inadvertent change or a bug in a pipeline back to the exact action that caused it. Users can run `DESCRIBE HISTORY` to see metadata around the changes that were made.

Delta Lake Transaction Log Summary

In this blog, we dove into the details of how the Delta Lake transaction log works, including:

- What the transaction log is, how it's structured, and how commits are stored as files on disk.
- How the transaction log serves as a single source of truth, allowing Delta Lake to implement the principle of atomicity.
- How Delta Lake computes the state of each table – including how it uses the transaction log to catch up from the most recent checkpoint.
- Using optimistic concurrency control to allow multiple concurrent reads and writes even as tables change.
- How Delta Lake uses mutual exclusion to ensure that commits are serialized properly, and how they are retried silently in the event of a conflict.

Interested in the open source Delta Lake?

Visit the [Delta Lake online hub](#) to learn more, download the latest code and join the Delta Lake community.

Articles in this series:

Diving Into Delta Lake #1: Unpacking the Transaction Log

Diving Into Delta Lake #2: Schema Enforcement & Evolution

Diving Into Delta Lake #3: DML Internals (Update, Delete, Merge)

Related articles:

[What Is A Data Lake?](#)

[Productionizing Machine Learning With Delta Lake](#)

Try Databricks for free

[Get Started](#)

Related posts

[Login](#)

August 21, 2019 by [Burak Yavuz](#), [Michael Armbrust](#) and [Brenner Heintz](#) in [Company Blog](#)

The transaction log is key to understanding Delta Lake because it is the common thread that runs through many of its most important...

[See all Company Blog posts](#)



[Why Databricks](#) ✓

[Product](#) ✓

[Solutions](#) ✓

[Resources](#) ✓

[About](#) ✓

Databricks Inc.
160 Spear Street, 13th Floor
San Francisco, CA 94105
1-866-330-0121



[See Careers
at Databricks](#)

[Login](#)

Spark and the Spark logo are trademarks of the Apache Software Foundation.

[Privacy Notice](#) | [Terms of Use](#) | [Your Privacy Choices](#) | [Your California Privacy Rights](#)

